

# ELI — Memory

ELI v2.1.91

August 2026

## Contents

|                                                                 |          |
|-----------------------------------------------------------------|----------|
| <b>ELI Memory Subsystem</b>                                     | <b>1</b> |
| Files . . . . .                                                 | 1        |
| The Memory class (memory.py) . . . . .                          | 1        |
| Schema ( $\approx$ 25 tables) . . . . .                         | 2        |
| recall_memory — the hybrid retriever (memory.py:1722) . . . . . | 2        |
| Vector store (vector_store.py) . . . . .                        | 3        |
| Knowledge graph (knowledge_graph.py) . . . . .                  | 3        |
| Truth layer (memory_truth.py) . . . . .                         | 3        |
| Weight decay & consolidation . . . . .                          | 3        |
| Honest assessment . . . . .                                     | 3        |
| Update — 2026-06-09 . . . . .                                   | 4        |

## ELI Memory Subsystem

eli/memory/ — 7.0k LOC, 13 files. The persistent substrate: relational + full-text + vector + graph, all local SQLite/FAISS. Companion to project\_overview.md.

### Files

| File                         | LOC   | Role                                           |
|------------------------------|-------|------------------------------------------------|
| memory.py                    | 4.7k  | the Memory god-class + DBPaths + module facade |
| knowledge_graph.py           | 643   | entity/relation graph (KG)                     |
| habits_memory_db.py          | 470   |                                                |
| vector_store.py              | 430   | FAISS vector index + embedder                  |
| __init__.py                  | 0     |                                                |
| system_index.py              | 278   | indexed apps/executables/files                 |
| memory_truth.py              | 190   |                                                |
| memory_adapter.py            | 131   | compat adapter                                 |
| memory_service.py,           | small | helpers/compat                                 |
| sqlite_memory.py, stores.py, |       |                                                |
| populate_memories.py         |       |                                                |

### The Memory class (memory.py)

A single metaclass-backed class ( $\sim$ 50 public methods) that owns essentially every persistent concern:

- **Semantic memory:** store\_memory, add\_memory, recall\_memory, search\_memory(ies), get\_recent\_semantic\_memory, adjust\_weight, apply\_weight\_decay.
- **Conversation:** add\_conversation\_turn, store\_conversation, get\_conversation\_history, get\_recent\_conversations, get\_recent\_turns\_since, search\_conversations, get\_turns\_for\_day, save\_session\_summary, get\_session\_summaries.
- **Habits:** log\_habit\_event, get\_habit\_events, add\_habit\_rule, get\_habit\_rules, record\_habit\_run.
- **Self-improvement / learning:** log\_learning\_event, log\_failure, log\_correction, add\_observation, log\_improvement, add\_capability\_proposal, propose\_capability, get\_pending\_proposals, get\_recent\_failures/improvements, add\_capability\_proposal, propose\_capability, get\_pending\_proposals, get\_recent\_failures/improvements.
- **Episodic/semantic/reflective aliases:** store\_episodic, store\_semantic, recall\_semantic, store\_reflective.
- **Stats / routing:** get\_stats, get\_dashboard\_counts, get\_db\_routing\_info.

vector\_store is a lazy property; the KG is integrated via knowledge\_graph.get\_knowledge\_graph().

## Schema (≈25 tables)

memories (+ legacy memory), conversation\_turns (+ conversations), session\_summaries, kg\_entities, kg\_relations, habit\_rules, habit\_events, habits, failures, corrections, improvements, observations, capability\_proposals, learning\_replay, user\_patterns, desktop\_apps, executables, recent\_files, user\_dirs, error\_tracking, recall\_log, events, semantic, emotion\_events. FTS5 virtual tables back conversation and KG search.

### emotion\_events (v2.1.31) — the emotional timeline

Owned by cognition/emotion\_timeline.py, not the Memory class: it opens user.sqlite3 directly (same pattern as habits\_memory\_db.py) and creates its table idempotently, so it adds nothing to the 4.7k-line god-class.

| column                      | meaning                                                                                                                                           |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| ts, user_id, session_id     | when / whose / which session                                                                                                                      |
| detected                    | what the USER seemed to feel                                                                                                                      |
| expressed                   | the register ELI answered in                                                                                                                      |
| valence                     | negative / positive / neutral — what makes “has been negative for a while” answerable without caring which specific emotion each read was         |
| confidence, source, arousal | how strong the read was and where it came from (voice / text / fused / override)                                                                  |
| user_text                   | the utterance that produced the read                                                                                                              |
| eli_prior_action            | <b>the action ELI ran on the PRECEDING turn</b> — this is the column that lets ELI ask whether the mood turned because of something <i>it</i> did |

Indexed on ts and (user\_id, ts). Reads come back ORDER BY ts DESC, id DESC — the id tiebreak matters because several reads can land inside one second and run-length counting would otherwise mis-order. See blueprints/perception.md for the assessment gates and the proactive surfacing path.

### recall\_memory — the hybrid retriever (memory.py:1722)

The shared retrieval foundation (see orchestration\_and\_agents.md for the two strategies on top):

1. **FAISS first** (Stage 5 vector primary). FTS5/LIKE runs only as a *supplement* when the vector index is empty/cold or returns < limit//2 hits. keyword\_only=True skips FAISS entirely (the orchestrator runs its own semantic\_search, so running FAISS here would double-search with a mislabeled source).

2. **Noise filtering** (important): excludes assistant\_insight/episodic/ reflection kinds, orchestrator source, and reflection/assistant\_insight/ session\_summary tags, and rows longer than 1500 chars — so ELI's own old responses/reflections never resurface as “recalled user memories”. This was the fix for the “Immutable Techniques” contamination class of bug.
3. **Importance-weighted ordering** via COALESCE(importance, 0.5).

Heavy inline column-detection (`_memory_table_columns`) guards against schema drift across versions — defensive, but a sign the schema has churned.

## Vector store (`vector_store.py`)

FAISS IndexFlat, embeddings via a local nomic embedder (`llama_cpp`). Notable: - `_embed_lock` (RLock) serializes embedding — the embedder is **not thread-safe** and concurrent calls segfault (this is why the orchestrator retrieval is sequential). - Metadata canonicalized to **meta.json** (migrated from legacy `meta.pkl`). - Singleton via `get_vector_store()`; shutdown-aware (skips embedding during teardown); `reset_vector_store()` for rebuilds.

## Knowledge graph (`knowledge_graph.py`)

`kg_entities(name,type,aliases,description,confidence) + kg_relations(subject_id, predicate, object_id, weight, source)` — a subject-predicate-object graph. FTS5 over entities (with insert/update/delete triggers) for fuzzy search\_entities. `upsert_entity`, `context_for_prompt` (lightweight SQLite-only prompt context — no embedding). Stop-word list prevents common words becoming entities.

## Truth layer (`memory_truth.py`)

`inspect_sqlite / inspect_vector_store` — read-only authority/inspection used by status surfaces; reads `vectors/meta.json` preferentially, falls back to legacy pickle. Backs `truth_report` and the memory-status surfaces.

## Weight decay & consolidation

`apply_weight_decay(decay_factor=0.98, min_weight=0.05, older_than_days=7)` — a multiplicative SQL update (`weight = MAX(min, weight*factor)` for old, low-importance rows). **There is no real episodic→semantic→KG consolidation**: decay is the only forgetting mechanism, and promotion across stores is manual (`store_episodic/store_semantic` are aliases, not a pipeline). This is a known gap ELI's own grounding admits.

## Honest assessment

- **Strong**: genuinely hybrid (vector + FTS5 + KG) on one local SQLite foundation; the noise-filtering in `recall_memory` is the right instinct and fixed real contamination; embedder serialization correctly avoids the segfault.
- **Weak**:
  1. `memory.py` is a **4.5k-line god-class** spanning ~8 unrelated concerns (semantic, conversation, habits, learning, failures, capabilities, system index). Wants to be split along those seams.
  2. **Schema sprawl / redundancy** — memories *and* memory, conversations *and* conversation\_turns, plus a standalone semantic table. The inline column-detection everywhere is compensating for schema instability.
  3. **No consolidation pipeline** — only multiplicative decay; memories don't graduate into the KG automatically.

## Update — 2026-06-09

- **Detected habits readable** (`Memory.get_detected_habits(min_count, limit)`): the proactive daemon fills the habits table via `detect_habits()`, but `HABIT_STATUS`, the persona overlay, and the bus `HabitAgent` all previously read only the (usually empty) `habit_rules` table → “no habits detected”. `get_detected_habits` reads the real habits table with a meta/introspection denylist (filters `SELF_REPORT/MEMORY_STATUS/EXPLAIN_*` noise from the user testing ELI), so genuine behaviour (media, app launches, screenshots) surfaces.
- **FAISS persistence bug fixed**: `_eli_persist_loaded_vector_store` was writing vector metadata with `pickle.dump` to `meta.json` while `_load_meta` reads JSON — so a rebuild corrupted the index and the next load silently auto-rebuilt (accumulating phantom vectors). Both write sites now use the canonical `_dump_meta` (JSON); the index re-syncs cleanly to the memory count. (This also closes a pickle-RCE-on-load vector the codebase had deliberately retired.) KG populator enriched (broader, clean entity extraction from `user_patterns`).